

Spilprogrammerings Workshop: Godot Version

Marcus Elmkvist, Toke Odgaard-Jans,
Henrik Prüss, Oliver Dahl Wagner

May 1, 2026

Contents

Chapter 1	Scene oversigt	Page 2
1.1	Setup af hoved scene(Scenes/Hoved_Scene/world.tscn)	2
1.2	Spiller Karakteren(Scenes/Spiller/player.tscn)	3
1.3	Spøgelse(Scenes/Spøgelse/ghost.tscn)	4
1.4	Shotgun(Scenes/ShotGun/shot_gun.tscn)	6
1.5	Kugle(Scenes/Kugle/bullet.tscn)	6
1.6	Spøgelse Spawner(Scenes/SpøgelseSpawner/spøgelse_spawner.tscn)	7
Chapter 2	Vejledning	Page 8
2.1	Spiller	8
2.2	Spøgelse	9
2.3	Shotgun	10
2.4	Kugle	13
2.5	Spawn Spøgelse	13
2.6	Score	14
	Valgfrie Opgaver — 16	
Chapter 3	Software Koncepter	Page 17
3.1	Modularitet	17
	Funktioner — 17 • Variabler — 17 • Instantiering — 18	
3.2	Software Patterns	18
	Object Pool — 18 • Observer — 19 • Finite State Machine — 19	
Chapter 4	Godot	Page 21
4.1	Godot Basics	21
4.2	Orchestrator	23

Chapter 1

Scene oversigt

1.1 Setup af hoved scene(Scenes/Hoved_Scene/world.tscn)

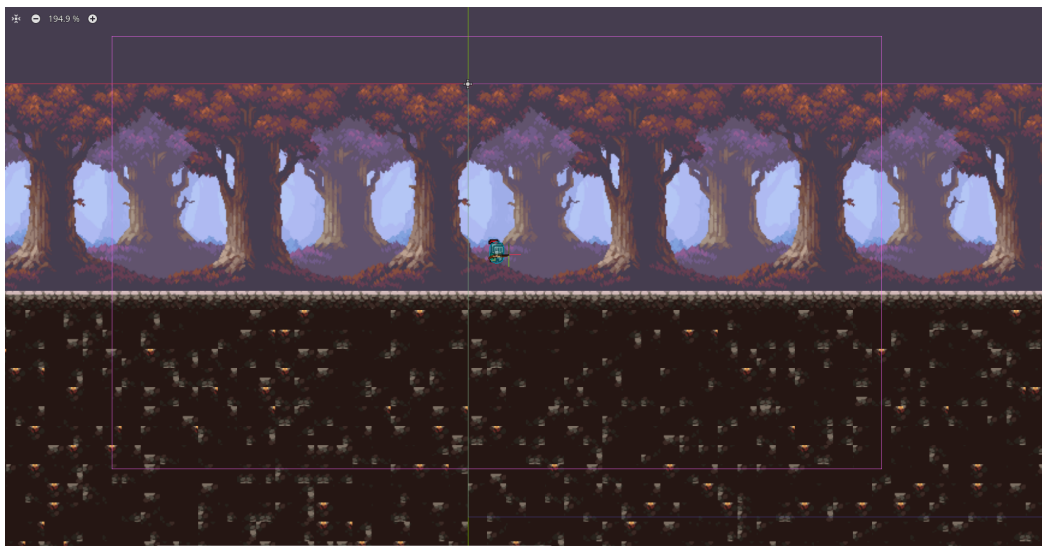


Figure 1.1: Main Scene

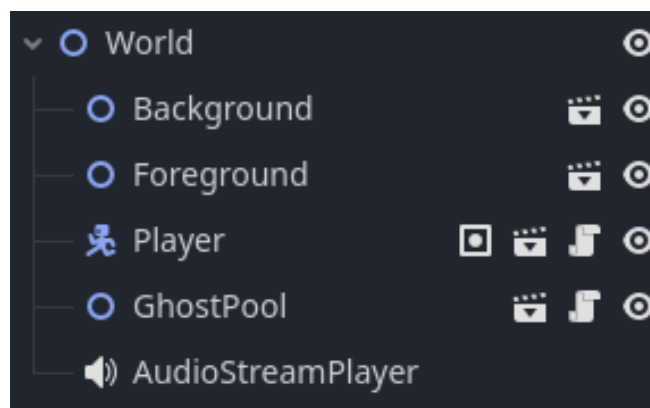


Figure 1.2: Main Scene Tree

- **World**

Hovednoden i hovedscenen og roden af hele nodetræet. **World** fungerer som overordnet container for alle

objekter i spillet og sikrer, at de deler samme koordinatsystem. Ved at samle spillets elementer under én fælles rod bliver det nemt at styre scene-skift, pausefunktioner og globale systemer.

- **Background**

Indeholder baggrundselementer, herunder parallax-lag. Denne opdeling adskiller baggrundens visuelle elementer fra gameplay-relevante elementer og gør det lettere at skabe dybde i scenen uden at påvirke fysik eller kollisioner.

- **Foreground**

Indeholder forgrundselementer som jorden og andre faste objekter, spilleren kan interagere med. Ved at samle disse elementer i én node skabes der en klar adskillelse mellem miljø og aktører, hvilket gør kollision-sopsætning og niveau-design mere overskueligt.

- **Player (CharacterBody2D)**

Repræsenterer spillerens karakter i spillet. Denne node håndterer spillerens bevægelse, input og interaktioner med omgivelserne. At placere spilleren direkte i hovedscenen gør det nemt at styre spillerens relation til niveauet som helhed.

- **GhostPool**

En separat container for alle spøgelsesfigurer. GhostPool anvendes til at styre generering og sværhedsgrad af fjender uden at blande denne logik direkte ind i hovedscenen. Denne opdeling gør det muligt at justere sværhedsgrad uafhængigt af banens opbygning.

- **AudioStreamPlayer2D**

Ansvarlig for afspilning af baggrundsmusik i scenen. At placere baggrundslyd i hovedscenen sikrer, at musikken er uafhængig af individuelle entities og fortsætter stabilt under gameplay.

1.2 Spiller Karakteren(Scenes/Spiller/player.tscn)

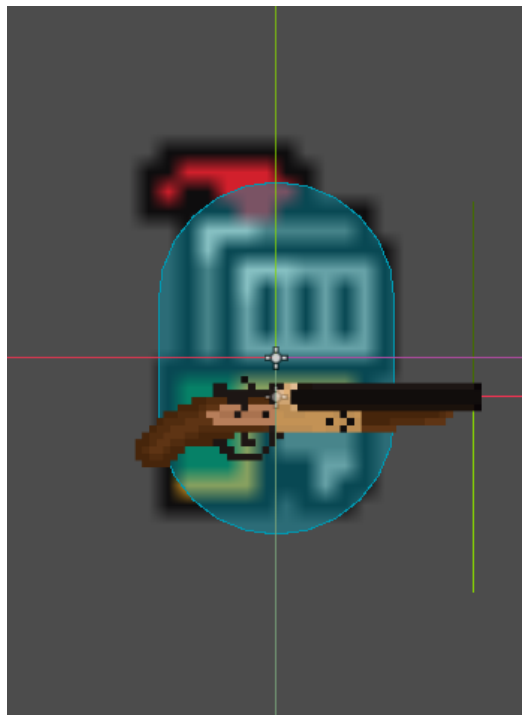


Figure 1.3: Player

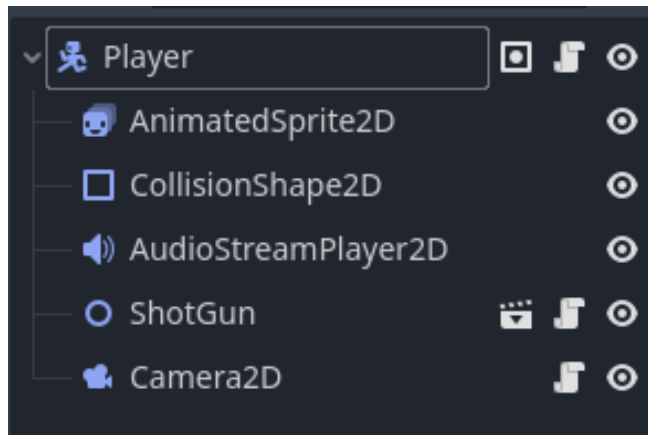


Figure 1.4: Player Scene Tree

- **Player (CharacterBody2D)**
Hovednoden for spillerkarakteren. Valget af **CharacterBody2D** giver adgang til indbygget understøttelse af bevægelse og kollision, hvilket forenkler implementeringen af spillerens fysik og respons.
- **AnimatedSprite2D**
Viser spillerens animationer.
- **CollisionShape2D**
Definerer spillerens kollisionsform. Gør det muligt for spøgelse at interagere korrekt i forbindelse med spillerens handlinger.
- **AudioStreamPlayer2D**
Spiller lydeffekter for spøgset, f.eks. lyde ved angreb eller når det bevæger sig.

1.3 Spøgelse(Scenes/Spøgelse/ghost.tscn)

- **Ghost (CharacterBody2D)**
Hovednoden for spøgelsesfjenden. Denne node repræsenterer ét individuelt spøgelse i spillet og håndterer dets bevægelse og interaktion med omgivelserne. Valget af **CharacterBody2D** gør det muligt at bevæge spøgset kontrolleret i forhold til spiller og miljø uden at anvende fuld fysiksimulering.
- **AnimatedSprite2D**
Viser spøgelsens animationer, fx svævende bevægelse eller angreb. Ved at holde animation adskilt fra logikken kan det visuelle udtryk nemt justeres uden at påvirke adfærden.
- **CollisionShape2D**
Definerer spøgelsens kollisionsområde. Denne bruges både til at registrere kontakt med spilleren og til at afgøre, hvornår spøgset rammes af kugler.
- **Area2D**
Anvendes som et registreringsområde omkring spøgset, fx til at opdage spillerens position. Dette gør det muligt for spøgset at reagere på spilleren uden tæt kobling mellem deres scripts.
- **AudioStreamPlayer2D**
Afspiller lydeffekter for spøgset, såsom svævelyde eller lyde ved angreb og død. Da lydnoden er et barn af spøgset, følger lyden automatisk med dets position i verdenen.

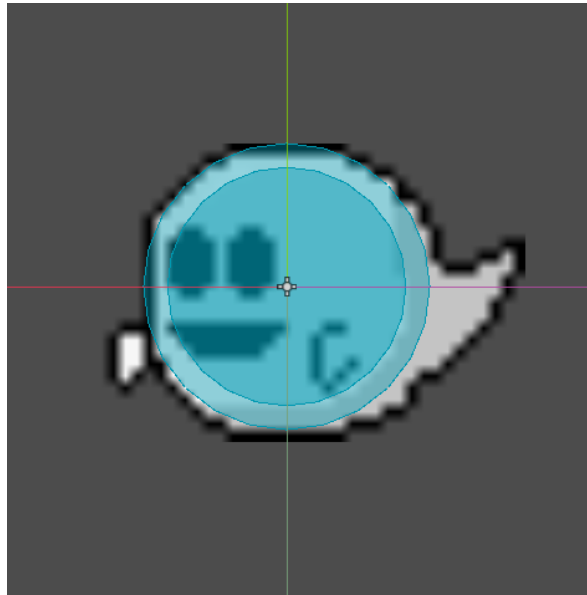


Figure 1.5: Spøgelse

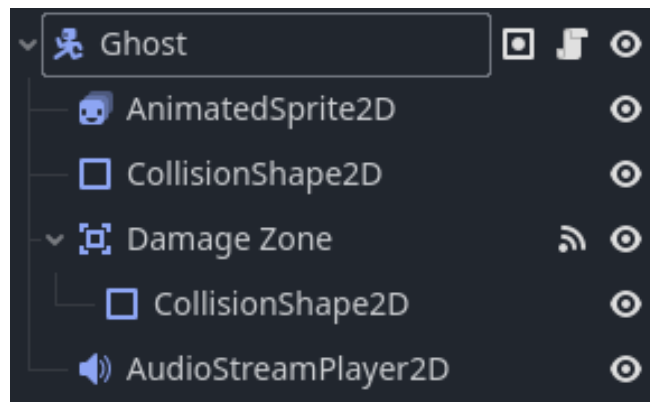


Figure 1.6: Spøgelses Scene Tree

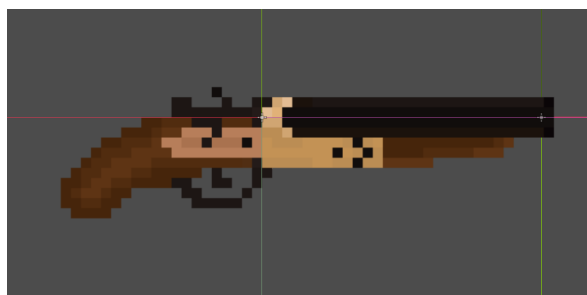


Figure 1.7: Shotgun

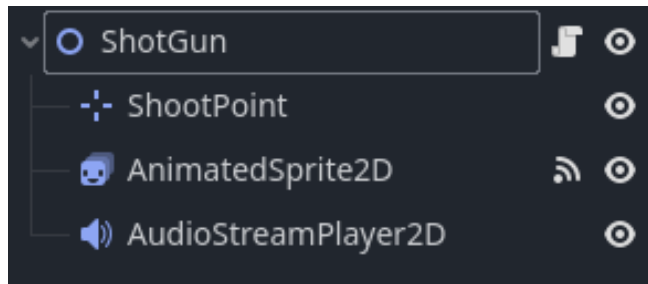


Figure 1.8: ShotGun Scene Tree

1.4 Shotgun(Scenes/ShotGun/shot_gun.tscn)

- **ShotGun**
Hovednoden, der repræsenterer shotgun-scenen. Samler alle relaterede elementer for våbnet.
- **ShootPoint**
Angiver, hvor projektiler skal affyres fra.
- **AnimatedSprite2D**
Viser animationer for shotgunen, som f.eks. recoil og affyring.
- **AudioStreamPlayer2D**
Spiller lydeffekter relateret til shotgunen.

1.5 Kugle(Scenes/Kugle/bullet.tscn)

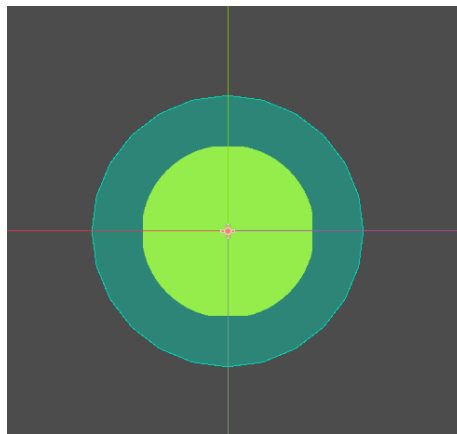


Figure 1.9: Bullet

- **Bullet (Area2D)**
Hovednoden for kuglen. **Area2D** bruges, da kugler primært skal registrere overlap med fjender og ikke reagere med fuld fysik.
- **Sprite (Bullet)**
Viser kuglens visuelle repræsentation i spillet.
- **CollisionShape2D**
Definerer kuglens kollisionsområde og bruges til at afgøre, hvornår den rammer en fjende.
- **PointLight2D**
Tilføjer dynamisk lys til kuglen, hvilket gør dens bevægelse mere tydelig og visuelt interessant.

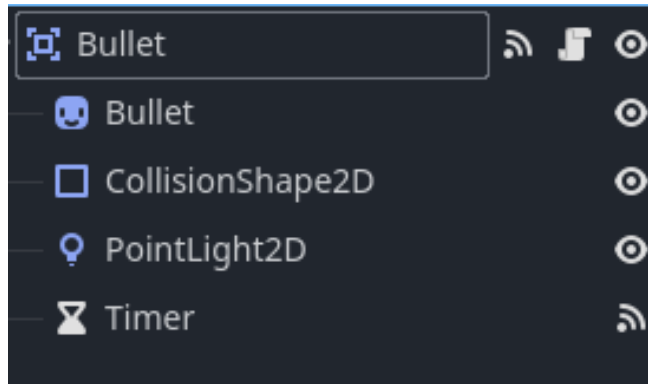


Figure 1.10: Bullet Scene Tree

- **Timer**

Styrer kuglens levetid, så den automatisk fjernes efter en vis tid. Dette forhindrer unødvendige objekter i at ophobe sig i scenen.

1.6 Spøgelse Spawner(Scenes/SpøgelseSpawner/spøgelse_spawner.tscn)

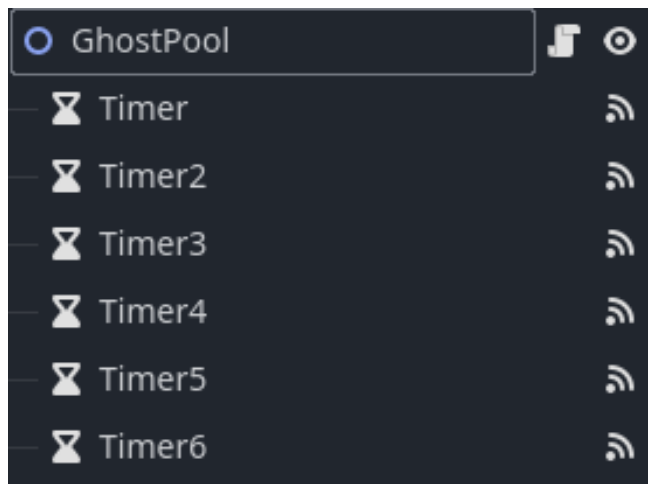


Figure 1.11: GhostPool Scene Tree

- **Spøgelse_spawner**

Overordnet styringsnode for alle spøgelse i spillet. Spøgelse_spawner fungerer som et simpelt sværhedsgradssystem, hvor antallet af aktive timere bestemmer hvor ofte nye fjender spawner.

- **Timer**

Bruges til at spawn spøgelse med faste intervaller.

- **Timer2–Timer6**

Ekstra timere der gradvist kan aktiveres for at øge spawn-frekvensen. Denne løsning gør det muligt at justere spillets sværhedsgrad uden at ændre selve spøgelses-logikken.

Chapter 2

Vejledning

2.1 Spiller

Opgave 1: Højre/Venstre-bevægelse

Lad os starte med at få vores karakter til at bevæge sig fra side til side. Som gennemgået i **Scene Overview**([Spiller Karakteren\(Scenes/Spiller/player.tscn\)](#)) er spillerens hovednode en **CharacterBody2D**, som er en indbygget node med funktionalitet, der gør det let at implementere bevægelse.

1. Åbn først spillerens script (**Scripts/Orchestrator/Spiller/player.torch**).
2. Til højre for event grafen kan du se lister for functions, macros, og variables.
3. Der er allerede variabler og metoder defineret for animationer — ikke fokus her, men gennemse dem efter behov.
4. Klik på + til højre for variables og tilføj en variabel kaldet **hastighed** (eller **speed**) som en integer og exportér den (tryk på øjet ved siden af), så den kan sættes i editoren. Tilføj også en variabel **max_hastighed** (eller **max_speed**) til senere brug.
5. I event-grafen: højreklik og tilføj en **PhysicsProcess**-node. Al bevægelsesfunktionalitet kaldes herfra.
6. Grundidé: tjek om venstre eller højre knap trykkes, og bevæg i den retning.
 - (a) Højreklik og tilføj to **Input Action**-noder til grafen.
 - (b) Vælg for hver **Input Action**-node hvilken knap den skal overvåge (brug f.eks. "højre" og "venstre" — piletasterne).
 - (c) Tilføj to **Branch**-noder (if-noder) og forbind boolean-outputtet fra hver **Input**-node til en **Branch**.
 - (d) Forbind **PhysicsProcess**-noden til den første **Branch**; forbind derefter den negative udgang (eller flow) til den næste branch efter behov.
 - (e) (Valgfrit) Placer **Print**-noder på de positive outputs for at teste: udskriv "Højre" eller "Venstre".
7. **CharacterBody2D** har en indbygget variabel **velocity** der bestemmer bevægelsesretning, den er af typen **Vector2**. Hvilket bare betyder at den har 2 værdi som bestemmer hvor meget den bevæger sig i x-aksen og i y-aksen.
 - (a) På hver branches positive output lav en ny **Vector2**, sæt x værdi til vores **hastighed**-variabel sæt **velocity** til at være den nye **Vector** ind ved hjælp af en **SetVelocity**-node (tilføj to, én per branch). Vælg "get **hastighed**" når du indsætter variabelen.
 - (b) For venstrebevægelse multiplicer værdien med **-1** for at bevæge mod negativ x-retning.

- (c) Hvis ingen input sæt `Velocity`'s x værdi til 0.
 - (d) Kald `move_and_slide` (eller tilsvarende f.eks. `move_and_collide` afhængigt af engine) for at udføre bevægelsen.
8. Nu har du en karakter, der kan bevæge sig til højre og venstre, gå til hoved scenen (`Scenes/Hoved.Scene/world.tscn`) for at teste din bevægelses implementation. Næste skridt: hop.
 9. (Valgfrit) Hov der er et eller andet som føles forkert i måden karakteren stopper på. Kan du finde ud af hvad det er og hvordan det kan fikses?
 10. (Valgfrit) Det fylder sørme meget nu, er der mon en måde hvorpå vi kan strukturere det så vi har en bedre oversigt over vores kode?

Opgave 2: Hop

Nu da vi kan bevæge os rundt skal vi have evnen til at hoppe ind i spil. Dette er relativt simpelt, vi skal bruge to ting:

1. Tyngdekraft
2. Når du trykker på en knap skal der tilføres en værdi til `velocity` y-aksen.

Lad os starte med tyngdekraften:

1. `CharacterBody2D` har en indbygget funktion som hedder `is_on_floor()` hvilket giver os en boolean værdi afhængigt af om vores karakters kollider rammer jorden. Tilføj et kald til dette i event-grafen.
2. Tilføj nu en branch som tjekke dette.
3. Hvis den er sand så gå bare videre i flowet. Ellers så tilgår vi den nuværende `velocity` og tilføjer i y-aksen en værdi på 9.82, og sætter så denne nye `Vector2d` til `velocity`.

Lad os nu få et hop implementeret:

1. Lav en ny integer variabel som hedder `jump_force` og sæt den til hvad du har lyst til, f.eks. 250.
2. Lav et nyt input tjek, på "Jump".
3. Tilføj en branch som tjekker den bool værdi, hvis sand så tilføjes denne værdi til `velocity`.

2.2 Spøgelse

Opgave 3: Bevægelse

Bevægelsen for spøgelset bruger Godots indbyggede `CharacterBody2D` og beskrives i trin nedenfor, i samme skrivestil som eksemplet.

1. Åbn spøgelsesnodens script (`Scripts/Orchestrator/Spøgelse/ghost.torch`).
2. Du vil se nogle eksisterende variabler eller metoder, som vedrøre animation, de er ikke fokuset her, men scan dem kort.
3. Tilføj en variabel til at holde reference til spilleren, kald den `player`, og sæt den til at være af typen `CharacterBody2D`. Sørg for at kunne sætte den fra editoren via export eller ved instansering.
4. Opret en eksporteret variabel `ghost_speed` (en float eller int) så hastigheden kan justeres i editoren.

5. I node-grafen tilføj en `PhysicsProcess` da det er her bevægelseslogikken køres.
6. Beregn retningen mod spilleren:
 - (a) Hent spøgelsets aktuelle position via `position`.
 - (b) Hent spillerens position via `player.position`.
 - (c) Træk spøgelsespositionen fra spillerpositionen og normaliser:
 - `direction = (player.position - position).normalized()`
7. Beregn velocity ved at gange retningen med hastigheden:
 - (a) `velocity = direction * ghost_speed`
8. Tildel velocity til `CharacterBody2D`'s indbyggede `velocity`-variabel.
9. Kald `move_and_slide()` inde i `_physics_process(delta)` for at bevare korrekt fysik og kollision.
10. Tilføj simple sikkerhedschecks:
 - (a) Hvis `player` ikke er sat, returner tidligt fra `_physics_process`.
11. Test i editoren: justér `ghost_speed`, sæt `player`-referencen, og verificér at spøgelse følger spilleren og håndterer kollisioner korrekt.

Opgave 4: Kollision med spiller

Lige nu sker der intet når spøgelse kolliderer med spilleren lad os korigere dette.

1. Som set ved Scene oversigt([section 1.3](#)) så har spøgelse en `Area2D` som vi kan bruge til at registre når spøgelse rammer spilleren.
2. Tilgå signal fanen^a, forbind signalet `body_entered` til Orchestrator grafen.
3. Her tjek at det er spilleren vi støder ind i.
4. For at holde det simpelt, lad os bare genstarte scenen:
 - (a) Tilgå rooten.
 - (b) Herefter kald metoden `reload scene`.

^aHvis du har brug for hjælp se [4.1](#)

2.3 Shotgun

Opgave 5: Kig efter mus

Lad os starte med at få shotgunen til at kigge efter musen.

1. Først lad os tilgå musens globale position. Dette er en statisk variabel som kan tilgås overalt.
2. Kald nu på metoden `Look.At()`, hvor du parser ind den globale position.
3. Shotgunen vil nu kigge mod musens position.

Opgave 6: Affyring

Nu da vi har shotgunnen til at sigte efter musen kan vi implementere det. Du ved allerede hvordan vi tilgår input så det går vi ikke igennem igen.

1. Tilføj og forbind den indbyggede Instantiering node.
2. vælg `bullet.tscn` som input til noden.
3. Tilgå positionen af bullet spawn noden.
4. Sæt kuglens position til spawn noden.
5. Kald `addSibling` metoden, og parse ind kuglen.

Du instantiere nu kuglen ind i scenen, men hov den bevæger sig ikke. Gå til [2.4](#) for at implementere kuglens bevægelse.

Opgave 7: Animations kontroller

Lige nu kan shotgun affyres lige så hurtigt, som spilleren kan trykke på knappen. Dette er underholdende, men ikke særligt realistisk for en shotgun. Derfor vil vi nu implementere en reload-mekanisme og samtidig sikre, at våbnet ikke kan skyde, mens det er i gang med at reloade.

Shotgunnen har tre animationer i sin `AnimatedSprite2D`:

1. Ready
2. Shoot
3. Reload

Disse tre animationer kan bruges som vores **states**. For enkelhedens skyld anvender vi tre bool-variabler: `Ready`, `Shooting` og `Reload`.

Opsætning af variabler

1. Opret en variabel i shotgun-scriptet kaldet `animation` og sæt typen til `AnimatedSprite2D`.
2. Opret en variabel kaldet `audio` og sæt typen til `AudioStreamPlayer2D`.
3. Opret en variabel kaldet `GunShot`, sæt typen til `AudioStream` og markér den som `export`.
I editoren: Vælg `ShotGun`-noden og tildel lyden `GunShot.wav` fra mappen `Assets/Sounds`.
4. Opret en variabel kaldet `ReloadSound`, sæt typen til `AudioStream` og markér den som `export`.
I editoren: Tildel lyden `Reload.wav` fra `Assets/Sounds`.

Initialisering

1. I `_ready()`-funktionen trækkes `AnimatedSprite2D`-noden fra Scene Tree ind i node-grafen og gemmes i `animation`-variablen.

Affyringslogik

1. Før input kontrolleres, skal der laves et tjek på `Ready`-state.
2. Hvis `Ready == true`, kontrolleres input for skydning.
3. Hvis input er aktivt:
 - Sæt `Ready` til `false`
 - Sæt `Shooting` til `true`

4. Kald `Play()` på `animation`-variablen med navnet `Shoot`.
5. Indsæt et delay på `0.2`.
6. Hent `audio`-variablen samt `GunShot`.
7. Kald `Set Stream` på `audio` og indsæt `GunShot`.
8. Kald `Play()` på `audio`.
9. Udfør herefter den eksisterende affyringslogik (instantiering af kugler).

Reload via `animation_finished`-signal

1. I Scene Tree: find `AnimatedSprite2D` og forbind signalet `animation_finished()` til shotgun-scriptet.
2. I den genererede metode:
 - Lav en `Branch`, som tjekker om `Shooting == true`.
3. Hvis sand:
 - Sæt `Shooting` til `false`
 - Sæt `Reload` til `true`
4. Afspil `Reload`-animationen via `animation.Play("Reload")`.
5. Indsæt et delay på `0.2`.
6. Sæt `audio-stream` til `ReloadSound` og kald `Play()`.
7. Hvis `Shooting == false`:
 - Lav en ny `Branch`, der tjekker om `Reload == true`.
 - Hvis sand:
 - Sæt `Reload` til `false`
 - Sæt `Ready` til `true`

Nu er shotgunen styret af tydelige states og kan ikke affyres, mens reload-animationen afspilles. Dette er et simpelt eksempel på en **Finite State Machine** baseret på bool-variabler.

Opgave 8: (Valgfrit) Knockback

Nu skyder shotgunen, men lad få lidt mere interesandte mechanics in over, lad os få den til at når vi skyder med shotgunen så bliver vi smidt i den modsatte retning.

- Lav en int variabel det hedder `KnockBackForce`.
- Tilgå spillerens `Charachterbody2D` via en `GetParent`, cast den derefter til en `Player`.
- Brug den samme logik fra tidligere til at finde hvor vi skyder mod.
- Normaliser denne `Vector` via `Normalize` funktionen.
- Gang(`Multiply`) denne `Vector` med din knockback variabel.
- Fra den tilgæede spiller karakter kald på `GetVelocity` og plus(`Add`) den med vores knockback vector.
- Kald nu `SetVelocity` på spilleren og sæt den til den nye samlede velocity vi lige lavede.

Tillykke du har nu implementeret en knockback effekt på din shotgun.

2.4 Kugle

Opgave 9: Kugler Bevægelse

Der er mange forskellige måder at implementere bevægelse. Helt essentielt vil vi gerne ændre kuglens globale (x,y) koordinater. Der er flere men lad os fokusere på 2 måder:

1. **MoveAlongXaxis**: Denne metode bevæger kuglen langs dets lokale x-akse. Få kuglen til at kigge mod hvor den skal bevæge sig det sted den bliver instantieret.
2. **Translate**: Denne metode forskyder kuglens position ved at tilføje et vektorforskydning til dens globale koordinater, uafhængigt af objektets rotation. Lav en variabel der hedder hastighed og exporter den.

Opgave 10: Kugler Kollision

For at dræbe spøgelse skal vi gøre lidt af det samme som vi gjorde for spøgelse tidligere.

1. Tilgå signal-fanen^a og forbind signalet `body_entered` til Orchestrator-grafen.
2. Tjek her, at det er spøgelse, vi støder ind i.
3. Hvis dette er tilfældet, fjerner vi spøgelse ved at kalde `queue_free()`.

^aHvis du har brug for hjælp, se 4.1

Opgave 11: Fjern Kuglen

Det giver ikke mening konstant at beregne bevægelse for en kugle, som ikke længere er på skærmen, så lad os fjerne den, når den er ude af synsfeltet. Få et signal fra on-screen-notifikation; når signalet modtages, startes en timer. Hvis kuglen ikke er kommet ind på skærmen igen inden timeren udløber, fjerner vi kuglen ved at kalde `queue_free()`.

2.5 Spawn Spøgelse

Opgave 12: Hvordan spawner vi spøgelse?

Der er to måder vi kan håndtere spawner og fjernelse af spøgelser:

- Instantiering: Lav en ny instans af spøgelser hver eneste gang den spawnes, og fjern den helt når den dør.
- Object Pooling: Hver gang der spawnes et nyt spøgelse tjekkes der først om der er et spøgelse allerede aktiv i hukommelse.
- lav en reference til spilleren og export den.

Spawn Spøgelse

Denne funktion sørger for at hente et spøgelse fra poolen eller oprette et nyt.

1. Instantiér `ghost.tscn`.

2. Sæt spøgelses `Pool`-reference til `self`.
3. Kald `SetPosition` for at placere spøgelse.
4. Tilføj spøgelse til scenen med `add_child`.

SetPosition-funktionen

Denne funktion placerer spøgelse i forhold til spilleren.

1. Opret funktionen `SetPosition`.
2. Hent spillerens `global_position`.
3. Brug `randi_range(-1, 1)` til at vælge retning.
4. Gang værdien med:
 - 330 på X-aksen
 - 180 på Y-aksen
5. Læg offset til spillerens position.
6. Sæt spøgelses `global_position`.
7. Sæt spøgelses `Player`-reference.

Timers (Spawn-kontrol)

1. Forbind hver timers `timeout()`-signal til `Pool`-scriptet.
2. I hver `timeout`-funktion:
 - Kald funktionen `GetGhost`.
 - Start timeren igen (eller lad næste timer tage over).

Opgave 13: (Valgfrit) Modulære Spøgelser

Nu har vi spøgelserne til at spawn. Lad os introducere lidt tilfældighed ind i spøgelserne. Andvend din spawn funktion til at få spøgelse til at bevæge sig med forskellige hastigheder og forskellige størrelser.

- Hint: Det er en function som hedder `randf_range`, som outputter en tilfældig værdi inden for en angivet grænse.

2.6 Score

Opgave 14: Score og Highscore med Save/Load

I denne opgave laver vi et simpelt score-system, der:

- tæller spillerens score løbende,
- gemmer den højeste score lokalt,
- indlæser highscore ved opstart af spillet.

Scoren opdateres via en `Timer`, og gemmes i en fil ved lukning af spillet.

Opsætning af variabler

1. Vedhæft et script til `PanelContainer`.
2. Opret følgende `@onready`-variabler:
 - `score` af typen `Label`, sat til `Value`.
 - `high_score` af typen `Label`, sat til `HighScoreValue`.

Score-opdatering via Timer

1. Forbind timerens `timeout()`-signal til scriptet.
2. I `timeout`-funktionen:
 - Konvertér `score.text` fra `String` til `int`.
 - Læg 1 til scoren.
 - Konvertér værdien tilbage til `String`.
 - Opdatér `score.text`.

Load ved opstart

1. Brug `_ready()`-funktionen.
2. Kald funktionen `load_game`.

Save ved lukning

1. Brug `_exit_tree()`-funktionen.
2. Kald funktionen `save_game`.

SaveGame-funktionen

1. Opret funktionen `save_game`.
2. Brug `FileAccess.open` med stien:

```
user://savegame.save
```

3. Åbn filen i `WRITE`-mode.
4. Kald funktionen `save` for at hente data.
5. Konvertér data til JSON via `JSON.stringify`.
6. Skriv JSON-strengen til filen.

LoadGame-funktionen

1. Opret funktionen `load_game`.
2. Tjek om save-filen eksisterer med `FileAccess.file_exists`.
3. Hvis ikke, returnér tidligt.
4. Åbn filen i `READ`-mode.
5. Læs linjen fra filen.

6. Brug `JSON.parse` til at indlæse data.
7. Hvis parsing fejler, udskriv en fejl og returnér.
8. Hent `HighScore` fra JSON-data.
9. Sæt `high_score.text`.

Save-funktionen

1. Opret funktionen `save`.
2. Returnér en dictionary med:
 - `"HighScore"` sat til den højeste score.

Highscore-beregning

1. Opret funktionen `ReturnHighscore`.
2. Sammenlign `score.text` og `high_score.text`.
3. Returnér den højeste værdi som `int`.

2.6.1 Valgfrie Opgaver

Opgave 15: Dash

Lige nu kan vores spiller figur kun gå rundt og hoppe. Lad os få lidt mere hastighed ind i ham. Implementer en funktionalitet som gør at når du trykker på en knap så bevæger han sig hurtigt i en retning i et kort stykke tid.

Opgave 16: To skud

Shotgunen kan lige nu kun skyde en gang før den er nødt til at reloade. Dette giver ikke visuelt mening da det er en to cyliner shotgun. Implementer det sådan at shotgunen kan skyde to gange før det behøver at reloade.

Chapter 3

Software Koncepter

3.1 Modularitet

Modularitet handler om at opdele et program i mindre, selvstændige dele som står for sine egne dele. Formålet er at gøre kode lettere at forstå, vedligeholde og genbruge. Når et system er modulært, kan enkelte dele ændres uden at påvirke resten af programmet unødvendigt.

3.1.1 Funktioner

Definition 3.1.1: Funktioner

“En funktion er en afgrænset blok kode, som udfører en bestemt opgave og kan kaldes flere gange fra forskellige steder i programmet.”

3.1.1 Hvad er det?

Funktioner bruges til at samle logik for at gentage den samme kode flere gange. Hvis et program for eksempel skal beregne skade i et spil forskellige steder, giver det mere mening at lave en funktion til al den beregning end at skrive den samme formel igen.

3.1.2 Hvordan?

En funktion oprettes med et navn, med parametre (hvis den skal have det) og en returnværdi (hvis den skal have det). Når funktionen kaldes, udføres dens kode. Parametre gør det muligt at sende data ind i funktionen, og returnværdien gør det muligt at få et resultat tilbage.

Ved at opdele et program i flere små funktioner bliver koden mere overskuelig, lettere at teste og nemmere at genbruge i andre dele af systemet.

3.1.2 Variabler

Definition 3.1.2: Variabler

“Et variabel er et navngivet område i hukommelsen, som bruges til at gemme data, der kan anvendes og ændres under programmets kørsel.”

3.1.3 Hvad er det?

Variabler bruges til at holde styr på værdier i et program. Det kan være spillerens liv, en score, navnet på en bruger eller positionen af et objekt i verdenen. Uden variabler ville et program ikke kunne huske information mellem forskellige instruktioner.

3.1.4 Hvordan?

Et variabel deklarerer med en datatype(kommer an på kodesprog) og et navn. Datatypen bestemmer, hvilken slags data variabelen kan indeholde, for eksempel tal, tekst eller sand/falsk-værdier(boolean). Derefter kan variabelen tildeles en værdi og senere opdateres.

Ved at bruge meningsfulde navne og begrænse variabelers ansvar bliver koden lettere at læse og forstå.

3.1.3 Instantiering

Definition 3.1.3: Instantiering

“Instantiering er processen, hvor en klasse bruges som skabelon til at oprette et objekt af klassen i hukommelsen.”

3.1.5 Hvad er det?

I objektorienteret programmering beskriver en klasse, hvordan noget skal se ud og opføre sig, mens et objekt er en instanseret klasse i hukommelsen. Instantiering er derfor selve oprettelsen af objektet ud fra klassen.

En god analogi er en byggeplan til et hus. Tegningen beskriver huset, men først når huset bygges i virkeligheden, eksisterer den, som et objekt i vil i hukommelse.

3.1.6 Hvordan?

Når et objekt instantieres, reserveres der hukommelse til dets data, og klassens konstruktor kan bruges til at sætte startværdier. Derefter kan objektets variabler og funktioner anvendes gennem den oprettede instans.

Dette gør det muligt at skabe mange objekter ud fra samme klasse, hvor hvert objekt kan have sin egen tilstand, selvom de deler struktur og funktionalitet. Hvilke betyder hvis man ændre et objekts liv variable for eksempel, vil ændringen kun ske på objekt man henvendte sig til.

3.2 Software Patterns

En software architecture-mønster er eksempler på gode måder at strukturere kode på. De hjælper med at gøre systemer mere vedligeholdelige, skalerbare og genanvendelige ved at give velafprøvede løsninger på almindelige designproblemer.

3.2.1 Object Pool

Definition 3.2.1: Object Pool Pattern

“Improve performance and memory use by reusing objects from a fixed pool instead of allocating and freeing them individually[1].”

3.2.1 Hvad er det?

Object pooling er et softwaremønster som ligger inden for optimeringskategorien. En god analogi er en restaurant, hvor objekterne er tallerkener, som bruges til at servere retter til kunderne. Uden pooling ville restauranten smide tallerkenerne ud efter brug og bestille nye for hver eneste ret, der bestilles. Dette kan kun fungere, når restauranten har få kunder, da bestilling af nye tallerkener tager tid for leverancen, og afskaffelsen tager også tid, siden de skal køres væk. Dette er et massivt spild af ressourcer det giver langt mere mening at rengøre (nulstille) tallerkenerne og bruge dem igen til nye retter og kun bestille nye, når de er ved at løbe tør.

3.2.2 Hvordan?

En meget simpel måde at implementere **object pooling** på er ved at bruge to lister: den ene for aktive objekter og den anden for inaktive objekter. Den inaktive liste bliver ved opstart initialiseret til en bestemt størrelse. Når en funktionalitet i verden ønsker at “spawne” et nyt objekt, kalder den en metode i objekt-poolen, som tjekker, om der er objekter i den inaktive liste. Hvis der er nogen, tager den et fra listen, aktiverer det og flytter det til den aktive liste.

Når objektet skal returneres til poolen, kalder det enten selv en metode på poolen eller en anden funktion, der kalder den for det, hvor objektet bliver deaktiveret, nulstillet, fjernet fra den aktive liste og tilføjet til den inaktive liste.

3.2.2 Observer

Definition 3.2.2: Observer Pattern

“Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically[1].”

3.2.3 Hvad er det?

Observer er et design-/implementeringsmønster som har til formål at signalere til mange forskellige objekter i et en-til-mange forhold uden at lave stærke forbindelser imellem dem. Det bliver ofte brugt til at lave event-funktionalitet. Fundamentalt består det af to hovedkomponenter: en *observer* og et *subject*.

Et godt eksempel ville være spil-achievements: når spilleren udfører en handling (subject), informeres flere systemer (UI, statistik, belønninger) uden at disse systemer behøver at kende til hinanden.

3.2.4 Hvordan?

Observer-mønstret implementeres typisk ved at lade subjektet holde en liste af observers. Observers kan registrere (subscribe) eller afregistrere (unsubscribe) sig hos subjektet. Når subjektets tilstand ændres, kalder det en metode (fx `notifyObservers`) som itererer over listen og kalder en opdateringsmetode på hver observer, ofte med relevant data eller en reference til subjektet.

Godot: I Godot behøver vi ikke implementere det, da dette allerede er bygget ind i engineen i form af **signals(4.1)**. En node kan udsender et signal, og andre noder kan forbinde (connect) callback-funktioner til dette signal for at blive notificeret og opdatere sig automatisk.

3.2.3 Finite State Machine

Definition 3.2.3: State Machine

“Allow an object to alter its behaviour when internal state changes. The object will appear to change its class[1].”

3.2.5 Hvad er det?

State-mønsteret er en metode til at adskille forskellige opførsler inden for samme klasse. Det kan bruges overalt, hvor vi vil skifte opførsel baseret på tilstand. Eksempler: NPC-opførsel — state er en af de mest basale måder at implementere AI i spil; eller spillerkontroller, hvor du vil begrænse, hvilke input der kan registreres på et givet tidspunkt. Fx: ”Jeg kan ikke hoppe igen, når jeg allerede har hoppet.” eller ”Jeg kan ikke grave, når jeg ikke er på jorden.”

3.2.6 Hvordan?

Der er flere måder at implementere state-mønsteret på; her er tre almindelige tilgange med fordele og korte eksempler.

1) If-statements (boolske flag): **Beskrivelse:** Brug én eller flere boolske variabler eller kæder af if/else for at styre opførsel.

- **Fordele:** Simpelt at forstå og hurtigt at implementere for få tilstande.
- **Ulemper:** Bliver hurtigt uoverskueligt, når antallet af states eller overgange vokser.

2) Enums (enkelt state-variabel): **Beskrivelse:** Definér en enum med alle mulige states og skift en enkelt variabel mellem disse værdier.

- **Fordele:** Klarere oversigt over tilladte states; lettere at udvide end mange bools.
- **Ulemper:** Logikken for hver state kan stadig blive rodet, hvis implementeringen samles ét sted.

3) State-klasser (State Pattern / polymorfi): **Beskrivelse:** Implementér en base State-klasse/interface og lav en konkret klasse for hver tilstand. Objektet holder en reference til den nuværende state-instans og delegerer opførsel til den. Overgange håndteres ofte af state-objekterne selv.

- **Fordele:** Meget skalerbart og organiseret; enkel at teste og udvide; adskiller opførsel per state.
- **Ulemper:** Flere klasser og lidt mere boilerplate.

Valg af metode Valg afhænger af kompleksitet og vedligeholdelsesbehov: brug bools eller enums til simple systemer; brug state-klasser når adfærden og overgange er komplekse eller når du ønsker bedre separation og testbarhed.

Chapter 4

Godot

4.1 Godot Basics

4.1.1 Noder

I Godot er **noder** de grundlæggende byggesten i ethvert spil. Alt i en scene er en node: spillere, fjender, kameraer, lys, UI-elementer og logik. En node repræsenterer et enkelt ansvar, fx bevægelse, rendering eller kollision.

Der findes mange forskellige typer noder, som hver især har deres eget formål. Eksempler på ofte anvendte noder er:

- **Node** – den mest basale node uden visuel eller fysisk funktionalitet.
- **Node2D** – bruges til 2D-objekter med position, rotation og skalering.
- **CharacterBody2D** – en node med indbygget funktionalitet til spiller- og NPC-bevægelse med kollision.
- **Area2D** – bruges til at registrere overlap og triggere via signaler.
- **Sprite2D** og **AnimatedSprite2D** – bruges til at vise statiske eller animerede figurer.

Nodetræet Noder organiseres hierarkisk i et **nodetræ**. En scene består af én rodnoder med eventuelle børn, som igen kan have deres egne børn. Forældrerelationen bestemmer både struktur og adfærd:

- Hvis en forældrenode flyttes, følger alle dens børnenoder med.
- Børn kan tilgå deres forældre og søskende via nodestier.
- Funktionalitet kan opdeles logisk ved at give hver node et klart ansvar.

Denne struktur gør det let at opbygge komplekse objekter, fx en spiller bestående af bevægelse, animation, kollision og lyd, uden at samle alt i én klasse.

Scenes og genbrug En **scene** er et gemt nodetræ og kan fungere som både et færdigt niveau eller et genbrugeligt objekt (prefab). Scenes kan instantieres flere gange i andre scenes, hvilket gør Godot stærk i forhold til modularitet og genanvendelse.

4.1.2 Editor

Godot Editor er det centrale arbejdsredskab, hvor man opsætter scener, noder, scripts og spillets ressourcer. Editorens brugerflade er opdelt i flere områder, som tilsammen giver overblik over både struktur og funktionalitet.

Scene-panelet Scene-panelet (øverst til venstre) viser nodetræet for den aktuelle scene. Her kan du:

- Tilføje, fjerne og omorganisere noder.
- Se forældre–barn-relationer mellem noder.
- Vælge den node, du vil arbejde med.

Viewport Viewporten er det visuelle arbejdsområde, hvor scenen vises. Her kan du:

- Placere og flytte noder direkte i verdenen.
- Rotere og skalere objekter.
- Se spillets layout i real-time.

Inspector Inspector-panelet (højre side) viser egenskaberne for den valgte node. Her kan du:

- Redigere node-specifikke properties som position, hastighed og ressourcer.
- Sætte værdier på **export**-variabler fra scripts.
- Tilknytte scripts og ressourcer.

FileSystem FileSystem-panelet giver et overblik over projektets filer og mapper. Her kan du administrere:

- Scener (.tscn)
- Scripts
- Assets som billeder, lyd og animationer

Workflow Et typisk workflow i editoren er:

1. Opret en scene og tilføj relevante noder.
2. Strukturér nodetræet logisk.
3. Vedhæft scripts til noder for at tilføje adfærd.
4. Brug Inspector til at justere værdier og forbinde ressourcer.
5. Test scenen direkte i editoren.

4.1.3 Signals

Signals er en implementering af observer-mønsteret indbygget i Godot. Det er en defineret type, som kan gemmes i en variabel og videresendes til funktioner [2, 3]. Der er to måder at anvende signals på: via scripts eller via editoren. Lad os starte med at gennemgå deres anvendelse gennem editoren. Noder som **Button**, **CharacterBody2D** og **Area2D** har allerede indbyggede signaler klar til brug. Du finder dem ved i editoren at vælge den specifikke node, du er interesseret i, og klikke på fanen Signals i højre side af editoren (Figure 4.1).

For at forbinde et signal til en funktion i et andet node eller til samme node i editoren gør du følgende:

1. Vælg noden og åbn fanen **Signals** (se Figure 4.1).
2. Find det ønskede signal i listen og klik på **Connect...**
3. I dialogboksen vælger du målnoden (den node der skal modtage signalet), navnet på den metode der skal kaldes, og eventuelt hvilke argumenter der skal medsendes.
4. Tryk **Connect**. Editor genererer automatisk en tom metode i målnodens script.

Når signalet emitteres/udsendes (fx en knap trykkes), kaldes denne metode.

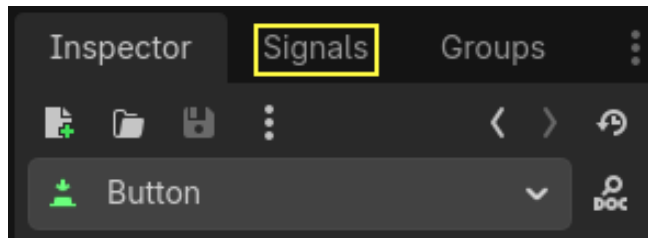


Figure 4.1: Fanen Signals

Afbryd og slet forbindelser:

- For at fjerne en forbindelse i editoren: åbn **Node**-panelet for målnoden, find forbindelsen under fanen **Connections** og klik **Disconnect**.
- I kode kan du afbryde en forbindelse med `disconnect()` hvis du har reference til forbindelsen.

Parameteroverførsel: Mange indbyggede signaler sender parametre (fx en body-reference fra `area_entered`). Stubs genereret af editoren inkluderer disse parametre, så du kan bruge dem direkte:

Oprette og sende egne signaler i script: I et script deklarerer du signaler med `signal`:
 Emitter/udsend signalet med `emit_signal()`:
 Andre noder kan så forbinde sig til dette signal via editoren eller med `connect()` i kode.

4.2 Orchestrator

Godot Orchestrator er et visuelt scripting-værktøj, hvor spillets logik opbygges ved at forbinde graf-noder fremfor at skrive traditionel tekstbaseret kode.

4.2.1 UI Navigation

Orchestrator har tre fundamentale UI områder: Event Grafen, funktioner og variabler. Den findes ved at trykke på *Orchestrator* til højre for *AssetLib* i Godots UI.

Event Grafen

Event Grafen er hovedarbejdsområdet i Orchestrator og fungerer som indgangspunkt for al logik i et script. Event grafen findes øverst til højre i *Orchestrators* UI. Her defineres, hvordan objektet reagerer på events, såsom opstart, input eller fysik-opdateringer.

Et *event* fungerer som startpunkt for et eksekverings-flow. Typiske events er:

- **Ready** – kaldes når noden er initialiseret og klar.
- **Process / PhysicsProcess** – kaldes hver frame.
- **Signal-events** – f.eks. tastetryk eller museklik.

I Event Grafen forbindes noder med to typer forbindelser:

- **Flow-forbindelser**, som angiver kørselsrækkefølgen.
- **Data-forbindelser**, som sender værdier såsom tal, bools eller objektreferencer.

Variabler

Variabler anvendes til at lagre data, som benyttes og ændres under programmets kørsel. Variabler oprettes centralt i et variabel-panel i højre side nederst i *Orchestrator* UI'en og kan tilgås fra både Event Grafen og funktioner.

Typiske variabeltyper inkluderer:

- Integer og Float (f.eks. hastighed, liv).
- Boolean (f.eks. tilstande som `isJumping`).
- Node-referencer (f.eks. reference til spilleren).

En variabel kan enten læses (**Get**) eller ændres (**Set**), hvilket tydeliggør dataflowet i grafen.

Script- og graf-oversigt

Script-området giver et overblik over det aktuelle Orchestrator-script og dets grafer. Her kan man:

- Skifte mellem Event Grafen og funktioner.
- Oprette nye funktioner.
- Navigere i scriptets struktur.

Dette område fungerer som et strukturelt overblik og kan sammenlignes med en fil- eller metodeoversigt i tekstbaseret kode.

Node-menuen

Node-menuen åbnes typisk via højreklik i en graf og indeholder alle tilgængelige noder. Dette inkluderer:

- Flow-kontrol (Branch, Sequence, Delay).
- Funktionkald til Godots API.
- Get/Set af variabler.
- Kald til brugerdefinerede funktioner.

Kan også åbnes via at trække en variabel ud fra en tidligere blok.

4.2.2 Graf Noder

Graf-noder er de grundlæggende byggeklodser i Orchestrator og repræsenterer handlinger, kontrolstrukturer eller dataoperationer. Hver node har et klart defineret formål og forbindes til andre noder for at danne et samlet eksekverings-flow.

En graf-node kan typisk bestå af:

- En eller flere **input-porte**, som modtager data eller et flow-signal.
- En eller flere **output-porte**, som sender data eller fortsætter eksekveringen.

Der skelnes primært mellem følgende typer af noder:

- **Event-noder**, som starter et flow (f.eks. `Ready`).
- **Flow-kontrol-noder**, som styrer programforløbet (f.eks. `Branch`).
- **Funktionsnoder**, som udfører en specifik handling eller beregning.
- **Variabel-noder**, som læser eller ændrer værdier.

4.2.3 Funktioner

Funktioner i Orchestrator bruges til at indkapsle og genbruge logik. En funktion består af sin egen graf og kan kaldes fra Event Grafen eller fra andre funktioner.

Brugen af funktioner understøtter modulær programmering og har flere fordele:

- Kompleks logik opdeles i mindre, overskuelige dele.
- Samme funktion kan genbruges flere steder i samme script.
- Event Grafen bliver mere læsbar og struktureret.

En funktion kan modtage input-parametre og returnere værdier, hvilket gør det muligt at arbejde struktureret med data. I Orchestrator vises funktioner som separate grafer, der kan navigeres via script- og graf-oversigten.

Funktioner kan oprettes på to måder. I højre side af Orchestrators UI findes en plus-knap, hvor nye funktioner kan tilføjes. Alternativt kan en gruppe af graf-noder markeres, hvorefter man kan højreklikke og vælge **Collapse to Function** i den fremkomne menu.

Bibliography

- [1] R. Nystrom, *Game Programming Patterns*. Genever Benning, Nov. 2014.
- [2] “Using signals — docs.godotengine.org.” https://docs.godotengine.org/en/stable/getting_started/step_by_step/signals.html. [Accessed 04-04-2026].
- [3] “Signal — docs.godotengine.org.” https://docs.godotengine.org/en/stable/classes/class_signal.html#class-signal. [Accessed 04-04-2026].